TrafficMAS+

Semafoare care negociază și coridoare pentru urgențe

Planul proiectului pentru materia **Sisteme Multi-Agent**
Echipă de 4 persoane · 10 săptămâni · Python, SimPy, NetworkX, OSMnx, Pygame

Cuprins

1. Conceptul și contribuția originală
2. Extinderea proiectului (varianta de bază vs. TrafficMAS+)
3. Arhitectura sistemului și contractele comune
4. Designul multi-agent (alinierea cu materia)
5. Rolurile echipei (4 membri)
6. Planul pe săptămâni și punctele de control
7. Structura repository-ului și modul de lucru
8. Evaluare și experimente
9. Riscuri și cum le evităm
10. Livrabile finale și cât de suficient este proiectul

1. Conceptul și contribuția originală

Fiecare intersecție își controlează propriile semafoare, dar comunică cu intersecțiile vecine ca să prevină formarea și propagarea ambuteiajelor. Vehiculele urmăresc o destinație și își adaptează ruta, iar vehiculele de urgență cer prioritate și pot reorganiza temporar traficul pe un coridor întreg.

Elementul original. Intersecțiile *negociază* timpul de verde în funcție de presiunea cozilor, printr-un protocol de tip licitație / Contract Net, cu un sistem de credite care asigură echitatea. O ambulanță nu doar forțează un singur semafor: ea *rezervă un coridor* pe ferestre de timp, intersecțiile din față își golesc cozile din timp, iar traficul revine la normal printr-o fază de recuperare controlată.

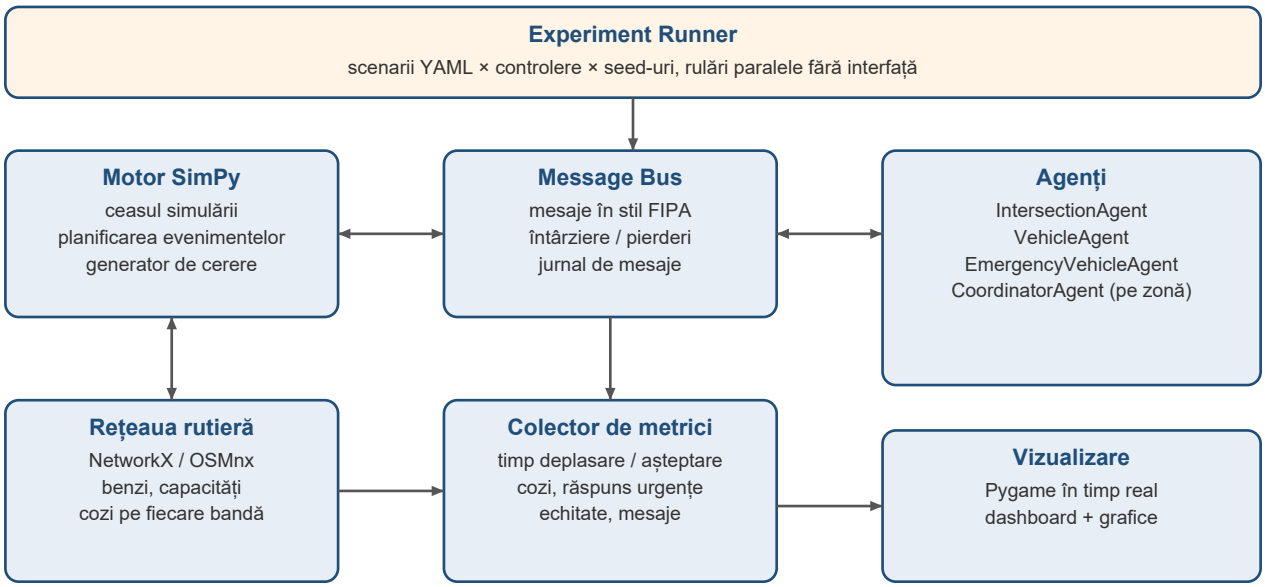
Ce evaluăm: timpul mediu de deplasare, timpul de așteptare, lungimea cozilor și timpul de răspuns al vehiculelor de urgență, la care se adaugă throughput-ul, echitatea (indicele Jain), emisiile estimate de CO_2 și costul comunicării.

2. Extinderea proiectului (varianta de bază vs. TrafficMAS+)

Varianta de bază (grilă artificială, 4 tipuri de agenți) se termină în 5–7 săptămâni. Extinderile de mai jos o transformă într-un proiect de 10 săptămâni, cu o încărcare reală și ușor de împărțit între 4 oameni.

Zonă	Varianta de bază	TrafficMAS+
Rețea	Grilă artificială	Grilă parametrică **și** un cartier real importat din OpenStreetMap (OSMnx), de ex. centrul Iașiului
Semafoare	Durată de verde adaptivă	Faze multiple, benzi de virare și 4 controlere interschimbabile: fix, actuat, max-pressure, **negociere**
Negociere	Prioritate după lungimea cozii	**Licitație / Contract Net**: intersecțiile licitează timp de verde după presiune; sistemul de credite păstrează echitatea
Coordonare	Broadcast de congestie	Coordinatori pe zone care detectează **spillback-ul** (blocajul care se propagă înapoi) și declanșează **unde verzi**
Vehicule	Rută fixă	A* cu costuri dependente de timp, **re-rutare** după congestia anunțată, profiluri de șofer
Urgențe	Cer prioritate	**Rezervarea coridorului** pe ferestre de timp, golirea cozilor din timp, recuperare controlată, rezolvarea conflictelor dintre ambulanțe
Perturbări	—	Accidente, benzi închise, cerere variabilă în timp (ora de vârf, matrice origine–destinație)
Comunicare	Apeluri directe	**Message bus** cu performative FIPA-ACL (REQUEST, PROPOSE, ACCEPT, REJECT, INFORM), cu întârziere și pierderi configurabile
Bonus (opțional)	—	Baseline cu învățare prin recompensă multi-agent (Q-learning independent), comparat cu negocierea
Evaluare	4 metrici	Cele 4 metrici + throughput, echitate, CO_2, număr de mesaje; teste statistice pe mai multe seed-uri

3. Arhitectura sistemului și contractele comune



Cheia lucrului eficient în paralel: interfețele se stabilesc în săptămâna 1 și se schimbă doar cu acordul echipei. Apoi fiecare lucrează pe implementări provizorii (stubs), deci nimeni nu așteaptă după altcineva.

```
# contracts.py -- agreed in week 1
@dataclass
class Message:
    sender: str
    receiver: str                # "*" = broadcast within the zone
    performative: Literal["REQUEST", "PROPOSE", "ACCEPT", "REJECT", "INFORM"]
    topic: str                   # "green_bid", "congestion", "corridor"
    payload: dict
    sent_at: float

class Agent(Protocol):
    id: str
    def on_message(self, msg: Message) -> None: ...
    def run(self, env: simpy.Environment) -> Generator: ...

class Controller(Protocol):      # fixed / actuated / max-pressure / negotiation / RL
    def next_phase(self, state: IntersectionState) -> tuple[int, float]: ...
```

4. Designul multi-agent (alinierea cu materia)

La Sisteme Multi-Agent se notează conceptele de *agent*, nu doar simularea. Elementele de mai jos fac aceste concepte explicite și trebuie să apară atât în cod, cât și în raport.

4.1 Proprietățile agenților (Wooldridge)

Agent	Autonomie	Reactivitate	Proactivitate	Abilitate socială
Intersecție	Își alege singură planul de faze	Reacționează la schimbarea cozilor și la cererile de coridor	Licitează timp de verde înainte să se umple cozile	Negociază cu vecinii, informează coordonatorul
Vehicul	Își alege singur ruta	Se re-rutează la congestie / incidente	Își urmărește destinația	Primește anunțurile de congestie
Vehicul de urgență	Își planifică coridorul	Replanifică dacă e blocat	Rezervă din timp intersecțiile din față	Cere și confirmă sloturile din coridor
Coordonator	Decide strategia zonei	Detectează spillback-ul	Lansează unde verzi	Agregă și transmite informații între zone

4.2 Arhitecturile agenților

- **IntersectionAgent — BDI simplificat.** Credințe: propriile cozi, starea vecinilor, coridoarele active. Dorințe: presiune minimă, servirea urgențelor, echitate. Intenții: planul de faze asumat, revizuit când apar credințe noi.
- **VehicleAgent — bazat pe utilitate.** Maximizează o utilitate formată din timpul estimat de deplasare și familiaritatea rutei, ponderată după profilul șoferului.
- **EmergencyVehicleAgent — planificator bazat pe obiective**, care replanifică atunci când o rezervare e respinsă sau drumul e blocat.
- **CoordinatorAgent — hibrid / pe niveluri.** Un nivel reactiv (alarme de spillback) și unul deliberativ (planificarea undelor verzi).

4.3 Protocoale de interacțiune

- **Negocierea timpului de verde:** FIPA Contract Net (cerere de propuneri → propuneri → acceptare / respingere) sau o licitație cu oferte închise, în care ofertele sunt valori de presiune plătite cu credite.
- **Rezervarea coridorului:** protocolul FIPA Request, cu sloturi pe ferestre de timp și un mesaj explicit de eliberare.
- **Informații despre congestie:** mesaje INFORM de tip publish/subscribe, limitate la zonă.
- Fiecare protocol apare în raport cu o **diagramă de secvență AUML**.

4.4 Organizare și aspecte de teoria jocurilor

- **Organizare ierarhică / holonică:** intersecțiile sunt grupate în zone, fiecare cu propriul coordonator.
- **Compatibilitatea stimulentelor:** bugetul de credite limitează supralicitarea strategică. Un experiment cu o intersecție „egoistă”, care își umflă ofertele, arată dacă mecanismul rezistă la manipulare.
- **Rutarea egoistă:** un scenariu dedicat reproduce **paradoxul lui Braess** și măsoară prețul anarhiei (cât costă re-rutarea egoistă față de cea coordonată).
- **Comportament emergent:** undele verzi care apar doar din negocierea locală, fără coordonator, sunt măsurate și vizualizate.
- **Framework opțional:** agenții rulează pe o buclă SimPy proprie; SPADE (bazat pe XMPP) poate fi folosit pentru mesagerie FIPA distribuită reală, dacă materia cere o platformă de agenți.

5. Rolurile echipei (4 membri)

Persoana 1 — Simulation Core & Network Architect

Construiește „lumea" în care trăiesc agenții.

- Motorul SimPy, ceasul simulării, generatorul de cerere (sosiri Poisson, matrice origine–destinație, profil de oră de vârf).
- Rețeaua rutieră: grila parametrică și importul OSM (OSMnx → NetworkX), benzi, capacități, viteze.
- Modelul de deplasare: o coadă pe fiecare bandă, cu timp de traversare și debit de saturație.
- Message bus-ul (întârziere, pierderi, jurnal de mesaje).
- Infrastructura repository-ului: structura, CI (GitHub Actions cu pytest și ruff), încărcarea configurărilor YAML.
- Ține și rolul de **integrator** în fiecare vineri.

Livrabile: `sim/, network/, bus/`

Persoana 2 — Intersection Control & Negotiation

Inteligența semafoarelor, adică elementul original al proiectului.

- IntersectionAgent: planul de faze, timpi minimi și maximi de verde, galben și all-red.
- Controlerele de referință: fix, actuat, max-pressure.
- **Protocolul de negociere:** fiecare intersecție calculează presiunea (coada care intră minus coada care iese) și licitează timp de verde; un vecin cedează dacă propria coadă e sub un prag; creditele împiedică o intersecție să câștige mereu.
- CoordinatorAgent pe zone: detectarea spillback-ului, sincronizarea offset-urilor pentru unde verzi.
- Opțional: baseline-ul cu învățare prin recompensă.

Livrabile: `agents/intersection.py, agents/coordinator.py, control/`

Persoana 3 — Vehicles, Routing & Emergency

Agenții mobili și scenariile cu urgențe.

- VehicleAgent: A* dependent de timp, re-rutare la mesaje INFORM despre congestie, profiluri de șofer.
- EmergencyVehicleAgent: calculează coridorul și trimite REQUEST corridor cu ETA pentru fiecare intersecție; intersecțiile își golesc cozile din timp, apoi urmează o fază de recuperare.
- Gestionarea conflictelor: două ambulanțe care se intersectează, cu prioritate după gravitate și ETA.
- Incidente: accidente și benzi închise care declanșează re-rutare; scenariul cu paradoxul lui Braess.

Livrabile: `agents/vehicle.py, agents/emergency.py, routing/, scenarios/incidents/`

Persoana 4 — Evaluation, Visualization & Experiments

Demonstrează că sistemul funcționează și cât de bine.

- MetricsCollector: timp de deplasare, timp de așteptare, lungimea cozilor, timp de răspuns la urgențe, throughput, indicele Jain, CO_2, număr de mesaje.
- Vizualizare live în Pygame: cozi colorate după lungime, starea semafoarelor, ambulanța și coridorul ei.
- Dashboard după rulare (Streamlit sau Matplotlib) pentru compararea controlerelor.
- Experiment runner: scenariu × controler × seed (de ex. 5 × 5 × 10 = 250 de rulări), în paralel cu multiprocessing.
- Analiza statistică (Mann–Whitney / Kruskal–Wallis, intervale de încredere) și raportul final.

Livrabile: `metrics/, viz/, experiments/, report/`

Responsabilități comune

- Fiecare pull request primește review de la cineva din **altă zonă**, ca toată lumea să înțeleagă întregul sistem.
- Fiecare își scrie testele (pytest) și documentația pentru modulele proprii.
- Ședință de 15 minute de două ori pe săptămână; integrare și demo în fiecare vineri.
- Fiecare membru scrie capitolul din raport despre zona lui; Persoana 4 le asamblează și le redactează.

6. Planul pe săptămâni și punctele de control

Săpt.	Obiectiv comun	P1 Core	P2 Control	P3 Vehicule	P4 Evaluare
1	Contracte și schelet	Repo, CI, contracts.py, ceas SimPy gol	Specificația protocolului de negociere (pe hârtie, AUML)	Specificația coridorului de urgență (AUML)	Definirea metricilor, formatul de log (CSV/Parquet)
2	Prima lume care rulează	Grilă 3×3, cozi, generator de cerere	Controler fix pe stub	Vehicul cu rută fixă (Dijkstra)	Logger, primul grafic cu cozi
3	**Vertical slice**	Message bus v1	Controler actuat	Re-rutare simplă	Pygame v1 (grilă, semafoare, mașini)
4	Negociere v1	Grilă N×M parametrică, configurări YAML	Max-pressure + negociere v1	Preempțiune pentru ambulanță la o intersecție	Experiment runner v1
5	Coordonare	Întârziere și pierderi în bus	Coordonator, detectarea spillback-ului	Coridor pe ferestre de timp	Metrici pentru urgențe și echitate
6	Rețea reală	Import OSM (cartier real)	Sistemul de credite pentru echitate	A* dependent de timp, profiluri de șofer	Dashboard v1
7	Perturbări	Profil de oră de vârf, matrice O–D	Unde verzi (offset-uri)	Accidente, conflicte între ambulanțe, scenariul Braess	Pygame pe harta OSM
8	**Feature freeze**	Optimizare (profiling, 1000+ vehicule)	Tuning de parametri / RL opțional	Scenarii de stres, testul cu agentul egoist	Campania completă de experimente
9	Analiză	Reproductibilitate (seed-uri, Docker)	Ablații (fără credite, fără coordonator)	Ablații (fără re-rutare)	Teste statistice, grafice finale
10	Livrare	README, ghid de instalare	Capitolul despre negociere	Capitolul despre urgențe	Raport, slide-uri, video demo

Puncte de control

- **Săptămâna 3 — vertical slice:** o simulare completă rulează cap-coadă, chiar dacă e simplă. Dacă nu e gata, săptămâna 4 se dedică doar integrării.
- **Săptămâna 5 — demo cu elementul original:** negociere plus o ambulanță care traversează un coridor rezervat.
- **Săptămâna 8 — feature freeze:** de aici încolo, doar reparații de buguri și experimente.
- **Săptămâna 10 — livrarea finală:** cod, raport, slide-uri, video demo.

7. Structura repository-ului și modul de lucru

```
TrafficMAS/
|-- trafficmas/
|   |-- contracts.py
|   |-- sim/  network/  bus/                        # P1
|   |-- agents/intersection.py  coordinator.py      # P2
|   |-- control/                                    # P2
|   |-- agents/vehicle.py  emergency.py  routing/   # P3
|   |-- metrics/  viz/                              # P4
|-- scenarios/      # grid_rush_hour.yaml, iasi_center_accident.yaml, braess.yaml ...
|-- experiments/    # runner + results/
|-- tests/
|-- report/
```

- Git flow: `main` rulează mereu; se lucrează pe branch-uri de feature; merge doar prin pull request cu review.
- CI rulează testele și lint-ul la fiecare pull request; un `main` stricat se repară înaintea oricărui alt lucru.
- Sarcinile se urmăresc pe un board GitHub Projects, cu o coloană pentru fiecare săptămână.
- Fiecare scenariu e un fișier YAML, iar fiecare rulare se poate reproduce din (scenariu, controler, seed).

8. Evaluare și experimente

Întrebarea centrală: cu cât e mai bună negocierea față de controlul fix, actuat și max-pressure, și cât îl costă pe restul traficului rezervarea coridorului pentru urgențe?

Metrică	Definiție	De ce contează
Timpul mediu de deplasare	Sosirea minus plecarea, pentru fiecare vehicul	Eficiența generală a rețelei
Timpul de așteptare	Timpul petrecut oprit în cozi	Experiența șoferului, calitatea semaforizării
Lungimea cozilor	Medie și maximă pe bandă; evenimente de spillback	Congestia și propagarea ei
Timpul de răspuns la urgențe	De la plecarea ambulanței până la destinație	Efectul rezervării coridorului
Costul perturbării	Întârzierea suplimentară a celorlalte vehicule după trecerea ambulanței	Prețul plătit pentru prioritate
Throughput	Vehicule care își termină traseul pe oră	Capacitatea sub încărcare
Echitate	Indicele Jain pe timpii de așteptare per direcție	Nicio direcție nu e neglijată
CO_2 (estimat)	Model de emisii pe timpul de oprire / ralanti	Impactul asupra mediului
Costul comunicării	Mesaje pe minut per agent	Scalabilitatea negocierii

Designul experimental

- **Controlere:** fix, actuat, max-pressure, negociere, negociere + coordonator (și RL opțional).
- **Scenarii:** cerere mică / medie / mare, oră de vârf, accident, bandă închisă, mai multe ambulanțe, rețeaua Braess, cartierul real din OSM.
- **Repetări:** minim 10 seed-uri pe combinație; rezultatele se raportează ca medie ± interval de încredere de 95%.
- **Statistică:** Kruskal–Wallis între controlere, apoi Mann–Whitney pe perechi, cu corecție pentru comparații multiple.
- **Ablații:** se elimină pe rând câte o componentă (credite, coordonator, re-rutare, golirea din timp a coridorului) ca să se vadă contribuția fiecăreia.
- **Robustețe:** se variază întârzierea și pierderile de mesaje din bus și se măsoară degradarea.

9. Riscuri și cum le evităm

Risc	Cum îl evităm
Integrarea întârzie	Contracte fixate în săptămâna 1, vertical slice până în săptămâna 3, integrare în fiecare vineri, nu la final.
Importul OSM e mai greu decât pare	E programat abia în săptămâna 6; grila parametrică rămâne mereu o variantă funcțională de rezervă.
Simularea e prea lentă pentru 250+ rulări	Deplasare pe cozi (fără fizică continuă), profiling în săptămâna 8, rulări fără vizualizare pentru experimente.
Volum de muncă inegal	Baseline-ul RL e opțional și îl preia cine are cel mai puțin de lucru; rolurile se reechilibrează în săptămâna 5.
Negocierea nu bate max-pressure	Tot e un rezultat valid: ablațiile și analiza explică de ce. Raportul se bazează pe dovezi, nu pe o victorie forțată.

10. Livrabile finale și cât de suficient este proiectul

- Codul sursă pe GitHub, cu CI, teste și README.
- Raport (aproximativ 25–35 de pagini): designul multi-agent, protocoalele cu diagrame AUML, experimentele, analiza statistică, concluziile.
- Slide-uri de prezentare și un video demo de 3–5 minute (vizualizarea Pygame a unei ambulanțe care traversează un coridor negociat).
- Pachet de experimente reproductibil (configurări + seed-uri + rezultate).

Este suficient pentru materia Sisteme Multi-Agent? Da. Ca dimensiune, proiectul e peste nivelul obișnuit pentru un proiect de echipă de 4 persoane. Acoperă agenți autonomi cu arhitecturi diferite, protocoale de interacțiune explicite (Contract Net, licitații, FIPA Request), o organizare ierarhică, comportament emergent și întrebări de teoria jocurilor (compatibilitatea stimulentelor, paradoxul lui Braess), toate susținute de o evaluare riguroasă. Condiția este ca secțiunea 4 să fie explicită în raport: la această materie se notează conceptele de agenți, nu doar o simulare de trafic care funcționează.